

Monte Carlo lokalizacija mobilnih robota

Matej Jurec

9. lipnja 2026.

Sadržaj

1	Uvod	3
2	Problem lokalizacije	4
2.1	Formulacija problema	4
2.2	Vrste lokalizacijskih problema	4
2.3	Markovljeva lokalizacija	4
3	Grid lokalizacija	6
3.1	Algoritam	6
3.2	Ograničenja grid lokalizacije	6
4	Monte Carlo lokalizacija	7
4.1	Reprezentacija uvjerenja pomoću čestica	7
4.2	Algoritam MCL	7
4.3	Svojstva MCL algoritma	8
5	Poboljšanja MCL algoritma	9
5.1	Slučajne čestice i oporavak od otmice robota	9
5.2	Adaptive MCL - KLD uzorkovanje	9
6	Model gibanja	10
7	Model mjerenja	10
8	Implementacija	11
8.1	Struktura programa	11
8.2	Tijek jednog koraka	11
8.3	Praktične prilagodbe	12
9	Ponašanje algoritma	13
9.1	Globalna lokalizacija i praćenje	13
9.2	Oporavak od otmice robota	13
10	Zaključak	14

1 Uvod

Autonomni mobilni roboti sve su prisutniji u svakodnevnom životu, od jednostavnih kućanskih uređaja poput robotskih usisavača, do autonomnih vozila i robotaksija koji se već polako koriste u pravom prometu. No svi ti robotski sustavi dijele isti temeljni problem, a to je precizno određivanje vlastitog položaja, a da pritom često ne znaju svoju početnu točku. Taj slučaj naziva se problem globalne lokalizacije.

Određivanje pozicije i orijentacije robota iz podataka njegovih senzora jedan je od temeljnih problema mobilne robotike. U ovom seminaru obradit ćemo dva algoritma koji ga rješavaju. Za razliku od Gaussovih tehnika obrađenih na nastavi, poput Kalmanovog filtra, ovi algoritmi pripadaju skupini neparametarskih filtara. Njihova je zajednička prednost što mogu raditi izravno sa “sirovim”, neobrađenim mjerenjima senzora i istovremeno predstaviti više mogućih hipoteza o poziciji.

Prvi algoritam je grid lokalizacija, koja prostor pozicija dijeli na rešetku ćelija i za svaku računa vjerojatnost da se robot u njoj nalazi. Drugi algoritam, na kojem je veći fokus, je Monte Carlo lokalizacija (MCL). MCL predstavlja distribuciju uvjerenja skupom nasumično uzorkovanih čestica, umjesto eksplicitnog opisa funkcije gustoće vjerojatnosti. U odnosu na Kalmanov filter ne zahtjeva linearizaciju modela gibanja i mjerenja ni pretpostavku o Gaussovom šumu, pa može raditi i s izrazito nelinearnim sustavima, a u odnosu na grid lokalizaciju troši znatno manje memorije i brže obrađuje nova mjerenja.

U sklopu implementacije prikazat ćemo oporavak od otmice robota, scenarija u kojem je robot premješten na nepoznatu lokaciju a da toga nije svjestan. No, u tom slučaju osnovni MCL algoritam zakaže, te ćemo implementirati proširenu verziju koja koristi slučajne čestice da uspije oporaviti i pronaći njegovu točnu poziciju.

2 Problem lokalizacije

Lokalizacija robota odgovora na pitanje: Gdje je robot sada? Bez odgovara na to pitanje robot nije u stanju donositi odluke o budućim akcijama. Problem lokalizacije možemo promatrati kao problem transformacije koordinata. Karta okoline opisana je u globalnom koordinatnom sustavu, a robot posjeduje vlastiti lokalni koordinatni sustav. Lokalizacija je proces uspostavljanja veze između ta dva koordinatna sustava.

2.1 Formulacija problema

Generalno, stanje robota u trenutku t opisano je vektorom

$$\mathbf{x}_t = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \quad (1)$$

gdje x i y predstavljaju položaj u ravnini, a θ orijentaciju robota. Imamo upravljački ulaz $u_{1:t}$ i trenutno mjerenje $z_{1:t}$, uz poznatu kartu okoline m . Cilj lokalizacije je procijeniti *uvjerenje* (engl. *belief*)

$$\text{bel}(\mathbf{x}_t) = p(\mathbf{x}_t \mid z_{1:t}, u_{1:t}, m) \quad (2)$$

odnosno distribuciju vjerojatnosti pozicije robota uz sve do sada prikupljene podatke.

2.2 Vrste lokalizacijskih problema

Postoje tri ključne vrste lokalizacijskih problema koji se razlikuju ovisno o tome koliko znamo o početnoj poziciji robota.

Kod **praćenja pozicije** robot zna svoju početnu poziciju i zadatak je samo pratiti njene promjene kroz gibanje. Uvjerenje je inicijalizirano uskom Gaussovom distribucijom oko poznate početne pozicije.

Kod **globalne lokalizacije** robot nema nikakvo znanje o svojoj početnoj poziciji pa uvjerenje počinje kao uniformna distribucija po cijelom prostoru. Robot mora sam odrediti gdje se nalazi isključivo na temelju mjerenja senzora, što je znatno teži problem jer distribucija uvjerenja može biti multimodalna, što znači da robot u isto vrijeme može imati više podjednako vjerojatnih hipoteza o svojoj lokaciji.

Najtežim slučajem smatra se **problem otmice robota**, gdje je robot premješten na nepoznato mjesto bez da je toga svjestan. Ovaj scenarij testira sposobnost algoritma da se oporavi od potpuno krive procjene pozicije, što je ključno za pouzdano djelovanje u uvjetima kakvi su u pravom svijetu.

2.3 Markovljeva lokalizacija

Markovljeva lokalizacija primjena je Bayesovog filtra na problem lokalizacije robota. Naziv dolazi od Markovljeve pretpostavke prema kojoj trenutno stanje robota ovisi isključivo o prethodnom stanju i upravljačkom ulazu, a ne o cijeloj povijesti gibanja. Algoritam se svodi na iterativnu primjenu dva koraka Bayesovog filtra na prostor pozicija robota. Korak predikcije:

$$\overline{\text{bel}}(\mathbf{x}_t) = \int p(\mathbf{x}_t \mid u_t, \mathbf{x}_{t-1}, m) \text{bel}(\mathbf{x}_{t-1}) d\mathbf{x}_{t-1} \quad (3)$$

Korak ažuriranja:

$$\text{bel}(\mathbf{x}_t) = \eta p(z_t \mid \mathbf{x}_t, m) \overline{\text{bel}}(\mathbf{x}_t) \quad (4)$$

Jednadžbe su identične onima koje smo izveli i dokazali na nastavi, jedina razlika je što je stanje sada pozicija robota, a ne stanje nekog fizičkog sustava, i to što algoritam pretpostavlja da je unaprijed zadana mapa prostora m .

3 Grid lokalizacija

Grid lokalizacija je konkretna, diskretizirana izvedba Markovljeve lokalizacije. Za razliku od Gaussovih filtera, može raditi sa sirovim podacima iz senzora i predstaviti višemodalna uvjerenja, pa rješava i problem globalne lokalizacije. Algoritam koristi histogram filter, kontinuirani prostor pozicija dijeli se na mnoštvo ćelija, i onda za svaku ćeliju posebno izračunava uvjerenje o tome koliko je vjerojatno da se robot u njoj nalazi.

3.1 Algoritam

Algoritam je sličan Bayesovom filtru. Uvjerenje je prikazano skupom diskretnih vjerojatnosti $\{p_{k,t}\}$ gdje svaka vrijednost $p_{k,t}$ odgovara jednoj ćeliji rešetke x_k . Algoritam prolazi kroz sve ćelije, model gibanja predviđa novu poziciju na temelju upravljačkog ulaza, a model mjerenja korigira uvjerenje ovisno o tome koliko dobro mjerenje senzora odgovara pojedinoj ćeliji. Model gibanja $p(\mathbf{x}_t | u_t, \mathbf{x}_{t-1})$ opisuje vjerojatnost da robot iz prethodne poze, uz zadani upravljački ulaz, dospije u novu pozu. Model mjerenja $p(z_t | \mathbf{x}_t, m)$ opisuje vjerojatnost dobivenog mjerenja senzora ako se robot nalazi u danoj pozi unutar poznate karte. Točka $\text{mean}(x_k)$ je centar ćelije x_k .

Algorithm 1 Grid lokalizacija

Require: $\{p_{k,t-1}\}, u_t, z_t, m$

```

1: for sve  $k$  do
2:    $\bar{p}_{k,t} = \sum_i p_{i,t-1} \cdot \text{motion\_model}(\text{mean}(x_k), u_t, \text{mean}(x_i))$ 
3:    $p_{k,t} = \eta \cdot \text{measurement\_model}(z_t, \text{mean}(x_k), m) \cdot \bar{p}_{k,t}$ 
4: end for
5: return  $\{p_{k,t}\}$ 

```

3.2 Ograničenja grid lokalizacije

Veličina ćelija ključan je parametar koji utječe na vrstu modela senzora, računalnu zahtjevnost i kvalitetu rezultata. Kod grube rešetke vrijednost modela mjerenja može se drastično razlikovati unutar jedne ćelije pa evaluacija samo u njezinom središtu daje loše rezultate. Sličan problem nastaje kod modela gibanja gdje robot može biti prespor da prijeđe iz jedne ćelije u drugu između dva koraka. Sitnije ćelije rješavaju problem točnosti ali povećavaju računalnu zahtjevnost. Na primjer, soba od 10×10 m sa ćelijama od 5 cm daje $\frac{10}{0.05} \times \frac{10}{0.05} = 200 \times 200 = 40\,000$ prostornih ćelija, a množenjem s 8 mogućih orijentacija dobivamo ukupno $40\,000 \times 8 = 320\,000$ stanja. U usporedbi s tim, MCL s $M = 1000$ čestica predstavlja smanjenje od više od 300 puta, što je jedna od prednosti njegovog korištenja.

4 Monte Carlo lokalizacija

Monte Carlo lokalizacija koristi skup nasumično uzorkovanih čestica za aproksimaciju uvjerenja o poziciji robota, pri čemu svaka čestica predstavlja jednu hipotezu o tome gdje bi robot mogao biti. Naziv dolazi od korištenja nasumičnog uzorkovanja kao temeljnog alata, a algoritam su prvi put predstavili F. Dellaert i njegovi suradnici 1999. godine. Za razliku od grid lokalizacije, MCL koncentrira čestice tamo gdje je vjerojatnost visoka, što omogućuje točniji prikaz s manjim brojem uzoraka.

4.1 Reprezentacija uvjerenja pomoću čestica

Uvjerjenje o poziciji robota prikazuje se skupom od M čestica:

$$\mathcal{X}_t = \left\{ \mathbf{x}_t^{[i]}, w_t^{[i]} \right\}_{i=1}^M \quad (5)$$

gdje $\mathbf{x}_t^{[i]}$ predstavlja pozu i -te čestice, a $w_t^{[i]}$ njezinu težinu. Skup čestica u svakom trenutku daje nam sliku o tome kako je vjerojatnost raspoređena po prostoru, što je temeljna ideja ove metode. Što je više čestica u nekom dijelu prostora, to je vjerojatnost da se robot tamo nalazi veća. Kod globalne lokalizacije čestice se na početku raspoređuju uniformno po cijelom prostoru jer robot nema nikakvo znanje o svojoj početnoj poziciji.

4.2 Algoritam MCL

Algoritam prima skup čestica iz prethodnog koraka $\mathcal{X}_{t-1}$, upravljački ulaz u_t , mjerenje senzora z_t i kartu prostora m .

Algorithm 2 Monte Carlo lokalizacija

Require: $\mathcal{X}_{t-1}, u_t, z_t, m$

```

1:  $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$ 
2: for  $m = 1$  to  $M$  do
3:    $x_t^{[m]} = \text{sample\_motion\_model}(u_t, x_{t-1}^{[m]})$ 
4:    $w_t^{[m]} = \text{measurement\_model}(z_t, x_t^{[m]}, m)$ 
5:    $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$ 
6: end for
7: for  $m = 1$  to  $M$  do
8:   uzorkuj  $i$  s vjerojatnošću  $\propto w_t^{[i]}$ 
9:   dodaj  $x_t^{[i]}$  u  $\mathcal{X}_t$ 
10: end for
11: return  $\mathcal{X}_t$ 
```

U prvoj petlji svaka čestica propagira se kroz model gibanja koji predviđa novu poziciju na temelju upravljačkog ulaza. Zatim se svakoj čestici dodjeljuje težina prema tome koliko dobro njezina pozicija odgovara primljenom mjerenju senzora. U drugoj petlji provodi se resampling, čestice se biraju prema njihovim težinama, čime čestice na vjerojatnim pozicijama preživljavaju, a one na pogrešnim pozicijama nestaju. Nakon dovoljno mjerenja čestice konvergiraju prema stvarnoj poziciji robota.

4.3 Svojstva MCL algoritma

MCL može aproksimirati gotovo svaku distribuciju od praktičnog značaja jer nije ograničen na parametarski oblik kao EKF. Točnost aproksimacije izravno ovisi o broju čestica M , više čestica daje točniji rezultat uz veće računalne zahtjeve. Prednost je što sami biramo M prema raspoloživim resursima, a uobičajena strategija je uzorkovati čestice sve dok ne stignu novi podaci u_t i z_t . Takva prilagodba resursima teško je ostvariva kod grid lokalizacije i Gaussovih tehnika.

Još je jedna od prednosti MCL-a sposobnost reprezentacije složenih multimodalnih distribucija, što mu omogućuje rješavanje problema globalne lokalizacije s visokom točnošću praćenja pozicije. Međutim, osnovna verzija algoritma ne može se oporaviti od tzv. ot-mice robota.

5 Poboljšanja MCL algoritma

Osnovna verzija MCL algoritma ima dva ograničenja, ne može se oporaviti od globalne pogreške u lokalizaciji (npr. otmice robota) i koristi fiksni broj čestica neovisno o trenutnoj nesigurnosti.

5.1 Slučajne čestice i oporavak od otmice robota

Kada je robot premješten bez upozorenja (*kidnapped robot problem*), sve se čestice nalaze na krivoj poziciji. Standardni MCL u tom slučaju ne može konvergirati jer resampling samo preraspoređuje postojeće čestice i ne može ih dovesti u dio prostora koji je ostao bez ijedne čestice. Rješenje bi bilo da se u svakom koraku ubaci mali udio nasumično raspoređenih čestica. Njihov broj nije fiksni, već se prilagođava praćenjem kratkoročnog i dugoročnog prosjeka težina:

$$w_{\text{slow}} += \alpha_{\text{slow}} (\bar{w}_t - w_{\text{slow}}) \quad (6)$$

$$w_{\text{fast}} += \alpha_{\text{fast}} (\bar{w}_t - w_{\text{fast}}) \quad (7)$$

gdje je $\bar{w}_t$ prosječna *nenormalizirana* težina čestica u trenutku t , uz uvjet $0 \leq \alpha_{\text{slow}} \ll \alpha_{\text{fast}}$. Bitno je da se $\bar{w}_t$ računa prije normalizacije težina, jer normalizirane težine uvijek imaju prosjek $1/M$ pa ne nose informaciju o tome koliko mjerenja uopće odgovaraju česticama. Udio slučajnih čestica tada iznosi:

$$p_{\text{rand}} = \max\left(0, 1 - \frac{w_{\text{fast}}}{w_{\text{slow}}}\right) \quad (8)$$

Kada mjerenja prestanu odgovarati česticama (kao nakon otmice), kratkoročni prosjek w_{fast} pada brže od dugoročnog w_{slow} , pa p_{rand} raste i algoritam počinje pretraživati cijeli prostor stanja. Čim se čestice ponovno usklade s mjerenjima, omjer se vraća prema jedinici i ubacivanje prestaje.

Alternativno rješenje bi bilo *Mixture MCL*, gdje se mali udio čestica (npr. 5%) generira direktno iz modela mjerenja umjesto iz modela gibanja. Time algoritam stalno predlaže čestice na lokacijama koje odgovaraju trenutnom mjerenju, neovisno o prijašnjem vjerovanju, što daje pouzdaniji oporavak po cijenu složenijeg uzorkovanja iz modela senzora.

5.2 Adaptive MCL - KLD uzorkovanje

Umjesto fiksnog broja čestica, broj čestica može se dinamički prilagoditi nesigurnosti. Kada su čestice skupljene oko jedne pozicije, potrebno ih je malo, a kada su raspršene potrebno ih je više. Temeljna ideja metode jest da se u svakoj iteraciji odabere onoliko čestica da, s vjerojatnošću $1 - \delta$, pogreška između prave posteriorne distribucije i njezine aproksimacije skupom čestica ostane manja od praga ε . Ta se pogreška mjeri Kullback-Leiblerovom divergencijom, po čemu metoda i nosi ime.

Potreban broj čestica iznosi

$$M_t = \frac{k-1}{2\varepsilon} \left(1 - \frac{2}{9(k-1)} + \sqrt{\frac{2}{9(k-1)}} z_{1-\delta}\right)^3 \quad (9)$$

gdje je k broj zauzetih ćelija rešetke nad prostorom stanja, ε dozvoljena pogreška aproksimacije, a $z_{1-\delta}$ kvantil normalne razdiobe. Metodu je uveo Dieter Fox i nije dio osnovnog

MCL-a, a ovdje se navodi kao poznato proširenje. U praktičnom dijelu seminara to nećemo implementirati, koristit ćemo fiksni M , ali svakako ostaje kao zanimljiva mogućnost nadogradnje ovog algoritma.

6 Model gibanja

Model gibanja $p(\mathbf{x}_t | u_t, \mathbf{x}_{t-1})$ opisuje vjerojatnost da robot iz prethodne poze, uz upravljački ulaz $u_t = [v, \omega]^T$, dospije u novu pozu. Koristi se velocity motion model, u kojem su stvarne brzine $\hat{v}$, $\hat{\omega}$ i dodatni rotacijski šum $\hat{\gamma}$ zašumljene verzije zadanih, uzorkovane iz trokutaste distribucije. Nova poza tada slijedi iz kinematike kružnog gibanja:

$$x' = x - \frac{\hat{v}}{\hat{\omega}} \sin \theta + \frac{\hat{v}}{\hat{\omega}} \sin(\theta + \hat{\omega} \Delta t) \quad (10)$$

$$y' = y + \frac{\hat{v}}{\hat{\omega}} \cos \theta - \frac{\hat{v}}{\hat{\omega}} \cos(\theta + \hat{\omega} \Delta t) \quad (11)$$

$$\theta' = \theta + (\hat{\omega} + \hat{\gamma}) \Delta t \quad (12)$$

U implementaciji je ovo funkcija `sample_motion_model_velocity`.

7 Model mjerenja

Model mjerenja $p(z_t | \mathbf{x}_t, m)$ opisuje koliko dobro mjerenje senzora odgovara danoj pozi. Senzor za svaki landmark mjeri udaljenost r i smjer ϕ , pa je vjerojatnost mjerenja umnožak Gaussovih gustoća pogrešaka udaljenosti i smjera preko svih landmark-a:

$$p(z_t | \mathbf{x}_t, m) = \prod_{j=1}^L \mathcal{N}(r_j; \hat{r}_j, \sigma_r^2) \mathcal{N}(\phi_j; \hat{\phi}_j, \sigma_\phi^2) \quad (13)$$

gdje su $\hat{r}_j$ i $\hat{\phi}_j$ predviđena udaljenost i smjer do j -tog landmark-a iz poze čestice. Čestice čija predviđanja dobro odgovaraju mjerenju dobivaju veliku težinu. U implementaciji je ovo funkcija `compute_weights`.

8 Implementacija

U implementaciji prikazujemo rad proširenog MCL-a kroz globalnu lokalizaciju robota i njegov oporavak nakon otmice. Simulirani prostor je dvodimenzionalan, a u njemu je nasumično raspoređeno $L = 7$ poznatih obilježja (landmark-a) unutar područja veličine 5×5 m. Robot kreće iz nasumične poze i giba se poznatim upravljačkim ulazom $u = [v, \omega]^T$, pri čemu je $v = 0,2$ m/s i $|\omega| = \pi/8$ rad/s, a predznak kutne brzine periodički se mijenja kako bi robot ostao unutar prostora. Vremenski korak je $\Delta t = 1/30$ s, a skup sadrži $M = 500$ čestica.

Korišteni parametri su sljedeći. U modelu mjerenja standardne devijacije šuma su $\sigma_r = 0,5$ m za udaljenost i $\sigma_\phi = 0,3$ rad za smjer. U modelu gibanja koristi se trokutasta distribucije s koeficijentima $\alpha_1 = \dots = \alpha_6 = 0,1$. Parametri proširenog MCL-a su $\alpha_{\text{slow}} = 0,02$ i $\alpha_{\text{fast}} = 0,3$, čime je zadovoljen uvjet $\alpha_{\text{slow}} \ll \alpha_{\text{fast}}$. Otmica robota dogodi se u koraku 170.

8.1 Struktura programa

Program je organiziran oko nekoliko funkcija:

- **sample_motion_model_velocity**: propagira sve čestice prema velocity motion modelu, uzorkujući šumne brzine iz trokutaste distribucije.
- **measure**: iz stvarne poze robota generira mjerenja, odnosno udaljenost i smjer prema svakom landmark-u.
- **compute_weights**: izračunava nenormalizirane težine čestica prema Gaussovom modelu mjerenja s udaljenošću i smjerom. Težine se računaju u logaritamskoj domeni radi numeričke stabilnosti.
- **low_variance_resample**: provodi resampling niske varijance, koji jednom nasumičnom početnom točkom i jednoliko raspoređenim intervalima daje konzistentniji odabir čestica od multinomijalnog resamplinga.
- **animate**: glavna petlja koja u svakom koraku pomiče robota, uzorkuje mjerenje, propagira čestice, računa težine, ažurira w_{slow} i w_{fast} te provodi prošireni resampling.

8.2 Tijek jednog koraka

Svaki korak slijedi standardnu MCL shemu, proširenu mehanizmom za oporavak. Najprije se sve čestice propagiraju modelom gibanja. Zatim se iz stvarne poze robota uzima mjerenje, a funkcija **compute_weights** svakoj čestici pridružuje nenormaliziranu težinu na temelju slaganja predviđenih i izmjerenih udaljenosti i smjerova prema landmark-ima.

Iz nenormaliziranih težina računa se njihov prosjek $\bar{w}_t$, koji je ključan za prošireni algoritam. Važno je da se prosjek uzima prije normalizacije, jer normalizirane težine uvijek imaju prosjek $1/M$ i ne nose informaciju o tome koliko mjerenja uopće odgovaraju česticama. Prosjekom se ažuriraju kratkoročni i dugoročni prosjek w_{fast} i w_{slow} , iz kojih se određuje udio slučajnih čestica

$$p_{\text{rand}} = \max\left(0, 1 - \frac{w_{\text{fast}}}{w_{\text{slow}}}\right). \quad (14)$$

U koraku resamplinga zatim se ubacuje $\lfloor M p_{\text{rand}} \rfloor$ čestica na nasumične poze jednoliko raspoređene po prostoru, dok se preostale čestice biraju funkcijom `low_variance_resample` proporcionalno težinama. Procjena poze robota u svakom je koraku sredina čestica, pri čemu se kut računa kružnim prosjekom.

8.3 Praktične prilagodbe

Uz osnovni algoritam dodane su tri prilagodbe. Čestice se inicijaliziraju uniformno po cijelom prostoru, čime se modelira potpuno neznanje o početnoj pozi. Nakon svakog resamplinga česticama se dodaje mali Gaussov šum, takozvani roughening sa standardnom devijacijom 0,03 m za poziciju i 0,05 rad za kut, kako bi se očuvala raznolikost čestica. Konačno, vrijednost p_{rand} dodatno se eksponencijalno izgladuje pa oporavak nakon otmice izgleda postupnije, što služi samo glađem prikazu i ne mijenja sam algoritam. Prvih nekoliko koraka prikazuje samo početno uvjerenje, odnosno uniformno raspoređene čestice i procjenu u sredini prostora, prije nego što stigne prvo mjerenje.

9 Ponašanje algoritma

Pokretanjem implementacije promatra se ponašanje proširenog MCL-a kroz tri faze: početnu globalnu lokalizaciju, stabilno praćenje i oporavak nakon otmice robota. Analiza je usmjerena na posljednju fazu, koja je i glavni motiv za korištenje proširene inačice algoritma.

9.1 Globalna lokalizacija i praćenje

Na početku su čestice raspoređene uniformno, pa je procjena, kao njihova sredina, smještena približno u središtu prostora i znatno udaljena od stvarne poze robota. Već nakon nekoliko desetaka koraka, kako pristižu mjerenja, čestice se skupljaju oko stvarne poze i procjena konvergira na pravu trajektoriju. Nakon konvergencije skup čestica stabilno prati robota, a udio slučajnih čestica p_{rand} ostaje blizu nule, jer mjerenja dobro odgovaraju česticama pa je w_{fast} približno jednak w_{slow} .

9.2 Oporavak od otmice robota

U koraku 170 robot se premješta na nasumičnu pozu bez upozorenja. Sve se čestice u tom trenutku nalaze na staroj, sada pogrešnoj lokaciji, pa njihova predviđena mjerenja prestaju odgovarati stvarnim. Posljedično naglo pada prosjek nenormaliziranih težina $\bar{w}_t$.

Upravo na taj pad reagira prošireni mehanizam. Kratkoročni prosjek w_{fast} , zbog velikog α_{fast} , pada znatno brže od dugoročnog w_{slow} , pa omjer $w_{\text{fast}}/w_{\text{slow}}$ postaje manji od jedan i p_{rand} raste. Algoritam tada počinje ubacivati slučajne čestice raspoređene po cijelom prostoru, čime ponovno pokriva i dio prostora u kojem se robot zaista nalazi. Kako neke od tih čestica padnu blizu nove poze i dobiju veliku težinu, resampling ih umnožava i skup se postupno ponovno skuplja oko ispravne lokacije. Čim se mjerenja opet usklade s česticama, prosjek težina raste, omjer $w_{\text{fast}}/w_{\text{slow}}$ vraća se prema jedinici, p_{rand} pada na nulu i ubacivanje prestaje.

Ovo ponašanje nam pokazuje zašto je proširenje algoritma nužno. Standardni MCL bez ubacivanja slučajnih čestica ostao bi zaglavljen na staroj lokaciji, jer resampling samo preraspoređuje postojeće čestice i ne može popuniti dio prostora koji je nakon otmice ostao potpuno bez čestica. Dodavanjem slučajnih čestica, čiji se broj automatski prilagođava preko w_{slow} i w_{fast} , algoritam dobiva sposobnost oporavka od globalne pogreške u lokalizaciji uz zanemariv trošak u razdobljima stabilnog praćenja.

10 Zaključak

Monte Carlo lokalizacija elegantna je primjena čestičnog filtra na problem lokalizacije mobilnih robota. Oslanjajući se na Bayesov filter, predstavlja distribuciju uvjerenja skupom čestica i time može riješiti globalnu lokalizaciju, što Gaussovim tehnikama poput Kalmanovog filtra nije moguće. Međutim, osnovni MCL ne uspijeva se oporaviti nakon otmice robota, dok ga proširenje sa slučajnim česticama pouzdano oporavlja, jer ubacivanjem novih čestica ponovno pokriva prostor koji je resampling ostavio prazan.

Glavne prednosti MCL-a u odnosu na grid lokalizaciju su linearna memorijska složenost, koncentracija čestica tamo gdje je vjerojatnost visoka te sposobnost oporavka od globalnih pogrešaka. S druge strane, algoritam traži unaprijed poznatu kartu okoline i osjetljiv je na odabir parametara modela gibanja i mjerenja.

U praksi je MCL danas dio standardnog ROS navigacijskog paketa `amcl` (Adaptive Monte Carlo Localization), koji se koristi na brojnim komercijalnim i istraživačkim platformama. Daljnja poboljšanja, poput KLD uzorkovanja koje dinamički bira broj čestica, te proširenje prema istovremenom kartiranju i lokalizaciji (SLAM), ostaju aktivno područje istraživanja.

Literatura

- [1] S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, MIT Press, 2005.
- [2] F. Dellaert, D. Fox, W. Burgard, S. Thrun, *Monte Carlo Localization for Mobile Robots*
- [3] S. Thrun, D. Fox, W. Burgard, F. Dellaert, *Robust Monte Carlo Localization for Mobile Robots*, 2001.
- [4] D. Fox, *KLD-Sampling: Adaptive Particle Filters*, MIT Press, 2001.